LAB SHEET

Experiment No. 2

Contrast Enhancement and Histogram-Based Image Processing using Python and OpenCV

Course / Subject	Computer Vision / Image Processing
Experiment	2
Name	_____
CUID / Roll No.	_____
Date	_____

Submitted as a practical record

Experiment No. 2

Experiment Name: Contrast Enhancement and Histogram-Based Image Processing using Python and OpenCV

Aim

To analyze and implement contrast enhancement techniques using histogram analysis and histogram equalization in Python and OpenCV to improve image quality and visual information in low-contrast images.

Purpose

Image contrast is important in computer vision. Poor illumination, sensor limitations, or environmental conditions can produce low-contrast images, making feature extraction and object recognition difficult. Histogram-based techniques improve the distribution of pixel intensities and can make image details easier to see. This experiment demonstrates contrast stretching, histogram analysis, global histogram equalization, and Contrast Limited Adaptive Histogram Equalization (CLAHE).

Prerequisites / Requirements

- Basic knowledge of digital image processing
- Python programming
- Libraries: OpenCV (cv2), NumPy, Matplotlib
- Understanding of grayscale images and pixel intensities
- Jupyter Notebook, Google Colab, or Visual Studio Code

Theory

Image histogram: a frequency distribution showing how many pixels have each intensity value. For an 8-bit grayscale image, intensities range from 0 (black) to 255 (white).
Contrast stretching: maps a selected input intensity range to a wider output range.
Histogram equalization: uses the cumulative histogram to redistribute intensity values across a broader range.
CLAHE: performs equalization on small tiles and limits contrast amplification to reduce excessive noise enhancement.

Procedure

1. Import OpenCV, NumPy, and Matplotlib.
2. Load a grayscale image (or convert a color image to grayscale).
3. Display the original image and plot its histogram.
4. Apply contrast stretching.
5. Apply global histogram equalization using OpenCV.
6. Apply CLAHE and compare the outputs.
7. Plot histograms of the original and enhanced images.
8. Compare brightness, contrast, and feature visibility; record observations.

Python Program

The following code assumes that a grayscale image named `low_contrast_image.jpg` is present in the working directory. Change the path to the image you want to process.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load an image in grayscale. Replace the path with your image file.
image = cv2.imread("low_contrast_image.jpg", cv2.IMREAD_GRAYSCALE)

# Check that the image loaded correctly
if image is None:
    raise FileNotFoundError("Check the image path and file name.")

# 1. Display original image and histogram
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.hist(image.ravel(), bins=256, range=(0, 256))
plt.title("Original Histogram")
plt.xlabel("Pixel intensity")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# 2. Contrast stretching (using the 2nd and 98th percentiles)
low, high = np.percentile(image, (2, 98))
if high > low:
    stretched = np.clip(
        (image.astype(np.float32) - low) * 255 / (high - low),
        0, 255
    ).astype(np.uint8)
else:
    stretched = image.copy()

# 3. Global histogram equalization
equalized = cv2.equalizeHist(image)

# 4. Contrast Limited Adaptive Histogram Equalization
clahe_operator = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
clahe_image = clahe_operator.apply(image)

# 5. Compare processed images
images = [image, stretched, equalized, clahe_image]
titles = ["Original", "Contrast Stretching",
          "Histogram Equalization", "CLAHE"]

plt.figure(figsize=(10, 6))
for i, (result, title) in enumerate(zip(images, titles), start=1):
    plt.subplot(2, 2, i)
    plt.imshow(result, cmap="gray", vmin=0, vmax=255)
    plt.title(title)
    plt.axis("off")
plt.tight_layout()
plt.show()

# 6. Compare their histograms
plt.figure(figsize=(10, 6))
for i, (result, title) in enumerate(zip(images, titles), start=1):
    plt.subplot(2, 2, i)
    plt.hist(result.ravel(), bins=256, range=(0, 256))
    plt.title(title + " Histogram")
    plt.xlabel("Pixel intensity")
    plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# 7. Save the enhanced images if required
cv2.imwrite("contrast_stretched.png", stretched)
cv2.imwrite("histogram_equalized.png", equalized)
cv2.imwrite("clahe_enhanced.png", clahe_image)

print("Contrast enhancement completed.")
print("Saved: contrast_stretched.png, histogram_equalized.png, clahe_enhanced.png")
```

Output and Observations

Illustrative output using a reproducible low-contrast grayscale test image. When running the program, the exact appearance and histogram depend on the input image selected.

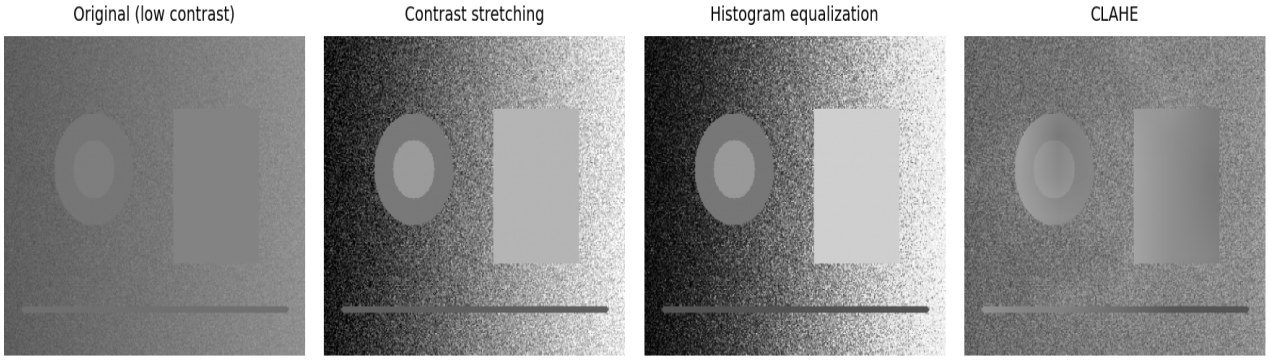


Figure 1. Original image compared with contrast stretching, global histogram equalization, and CLAHE.

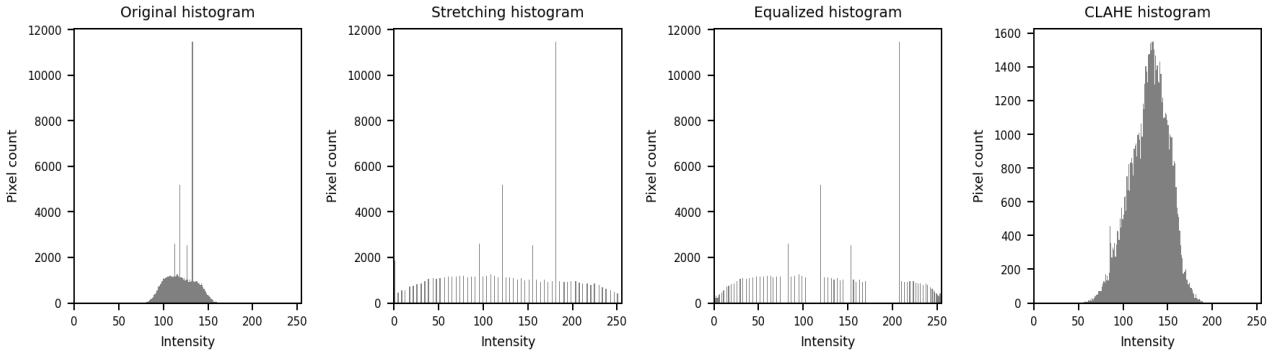


Figure 2. Pixel-intensity histograms for the original and enhanced images.

Output Analysis

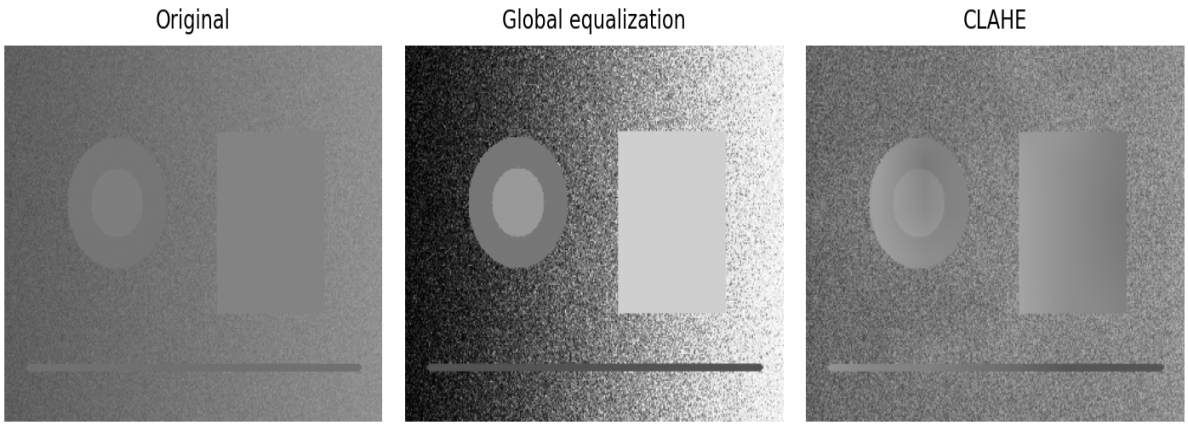


Figure 3. Visual comparison of the original image, global equalization, and CLAHE.

Observations

• The original image has a relatively narrow intensity range, so some details are difficult to distinguish.
• Contrast stretching expands the selected intensity range and increases the separation between darker and brighter regions.
• Global histogram equalization redistributes grayscale values across the image and can improve overall contrast, but may over-enhance noise or produce an unnatural appearance in some regions.
• CLAHE operates locally and clips the histogram to limit excessive contrast amplification. It often preserves local details better when illumination varies across an image.
• The best method depends on the image and the task; visual inspection and downstream task performance should both be considered.

Important note

The figures in this report are illustrative sample outputs, not screenshots captured from a live execution of the supplied question sheet. To reproduce the experiment, use your own image file and run the program in Python.

Answers to Questions

1. What is image contrast, and why is it important in image processing?

Image contrast is the difference in intensity or brightness between regions of an image. Good contrast makes objects, boundaries, and details easier to distinguish. It is important for visual interpretation and can support later tasks such as feature extraction, segmentation, and object recognition.

2. Explain the concept of an image histogram. What information does it provide?

An image histogram counts the number of pixels at each intensity value. For an 8-bit grayscale image, it covers intensity values from 0 to 255. It shows whether pixel values are concentrated in a narrow range, whether the image is generally dark or bright, and how intensity values are distributed. It does not show where those pixels occur spatially in the image.

3. Differentiate between Histogram Stretching and Histogram Equalization.

Histogram stretching linearly maps a selected input intensity range to a wider output range, increasing the dynamic range while largely preserving the ordering and relative spacing of intensities. Histogram equalization uses the cumulative distribution of pixel intensities to remap values non-linearly, aiming to spread frequently occurring intensities over a wider range. Stretching is direct and controllable; equalization can produce stronger but less predictable changes.

4. What is Contrast Limited Adaptive Histogram Equalization (CLAHE)? How does it differ from standard Histogram Equalization?

CLAHE divides an image into small regions (tiles), equalizes each region, and limits histogram peaks using a clip limit before combining the regions. Standard histogram equalization applies one global mapping to the whole image. CLAHE adapts to local illumination and limits excessive contrast amplification, so it can reveal local details while reducing the risk of amplifying noise too strongly.

5. Why is histogram equalization commonly applied before feature extraction and image segmentation?

It can increase the visibility of edges, textures, and intensity differences that may be hard to detect in low-contrast images. This can make some features easier to extract and some regions easier to separate. It is not guaranteed to improve every image, so results should be checked for noise amplification or loss of useful intensity relationships.

6. Mention three real-world applications where histogram equalization is widely used.

Three examples are: (1) medical image enhancement, such as improving visibility in radiographic images; (2) satellite and remote-sensing imagery, where contrast can help reveal terrain or land-cover features; and (3) low-light photography or surveillance images, where details may be hidden by poor illumination.

7. What are the limitations of global histogram equalization?

It treats the image globally and does not account for local illumination differences. It may over-enhance noise, make some regions too bright or too dark, and produce unnatural contrast. It can also alter intensity relationships that are meaningful for certain applications. CLAHE or other adaptive methods may be more suitable when local contrast varies.

8. How does contrast enhancement improve the performance of object detection and recognition systems?

By making object boundaries, textures, and distinguishing details more visible, contrast enhancement may help preprocessing, feature extraction, and segmentation. This can improve detection or recognition when poor contrast is the main difficulty. However, excessive enhancement can amplify noise or introduce misleading patterns, so performance should be evaluated on the target dataset.

9. Compare histogram-based enhancement techniques with brightness adjustment methods.

Brightness adjustment generally adds or subtracts an offset from pixel values, making the whole image lighter or darker while often leaving relative contrast similar. Histogram-based methods remap intensities according to their distribution, aiming to expand or redistribute contrast. Brightness adjustment is useful when an image is uniformly too dark or bright; histogram methods are more focused on improving intensity distribution and contrast.

10. Why is CLAHE preferred for medical imaging and low-light image enhancement?

CLAHE can improve local contrast in different regions without relying on a single global intensity mapping. Its clip limit helps control excessive amplification of noise, which is especially important in low-light images and medical images

where subtle details matter. It is not always the best method for every image; parameters and results should be reviewed carefully.

Result

Contrast enhancement techniques were studied using Python and OpenCV. Contrast stretching, global histogram equalization, and CLAHE were applied and compared using grayscale outputs and histograms. The comparison illustrates how histogram-based methods can improve the visibility of details in low-contrast images, while the choice of method depends on local illumination, noise, and the intended computer vision task.

Conclusion

Histogram analysis helps characterize an image's intensity distribution. Contrast stretching expands a chosen intensity range, global histogram equalization redistributes intensities across the image, and CLAHE adapts the process locally while limiting contrast amplification. Comparing both enhanced images and their histograms helps determine which method is most suitable for a particular image.